GPU Computing

Lecture 2: Architecture and CUDA Programming

Study notes integrating handwritten notes and lecture slides

Lecture of 22 October 2026

Abstract

This study guide integrates the handwritten notes for the second lecture with the slides *Lecture 02 – CUDA Programming*. It moves from the CPU/GPU comparison to the CUDA execution model, memory management, and the SAXPY example. It includes explanations, selected original figures, code examples, and review questions. Material from the first lecture contained in the notes PDF is excluded.

Contents

1 CPU and GPU: different architectural goals **2**

2 The CUDA programming model **3**

3 From indices to data: matrix addition **5**

4 Communication and synchronization **7**

5 The memory hierarchy **7**

6 Functions and compilation **9**

7 SAXPY: from the algorithm to total cost **11**

8 Common errors and a checking method **12**

9 Active review **13**

Sources and integration criteria **15**

1 CPU and GPU: different architectural goals

References: slides 3–7; handwritten notes pp. 5–6.

1.1 Latency and throughput

The CPU is designed to execute an instruction sequence quickly, including dependencies, branches, and irregular accesses. Caches and sophisticated control logic help reduce the completion time of a single thread: the objective is **latency minimization**.

The GPU devotes substantial resources to executing many threads in parallel. Its objective is **throughput**, the amount of work completed per unit of time. While some threads wait for memory, other ready threads can use the execution units. This is the meaning of *latency tolerance* in the handwritten notes: independent work hides the latency rather than eliminating it.

Aspect	CPU	GPU
Objective	Low latency for individual threads	High aggregate throughput
Resources	Sophisticated control and caches	Many processing units
Suitable work	Irregular flows, limited parallelism	Many elements with similar operations
Memory waits	Reduce and anticipate accesses	Switch between ready groups of threads

An effective cache can make a CPU access very fast; a cache miss may instead require access to main memory. GPUs also have caches: the comparison reflects different design priorities, not their absence. Moreover, many threads alone do not guarantee high performance: independent work, efficient accesses, and sufficient resources are also required.

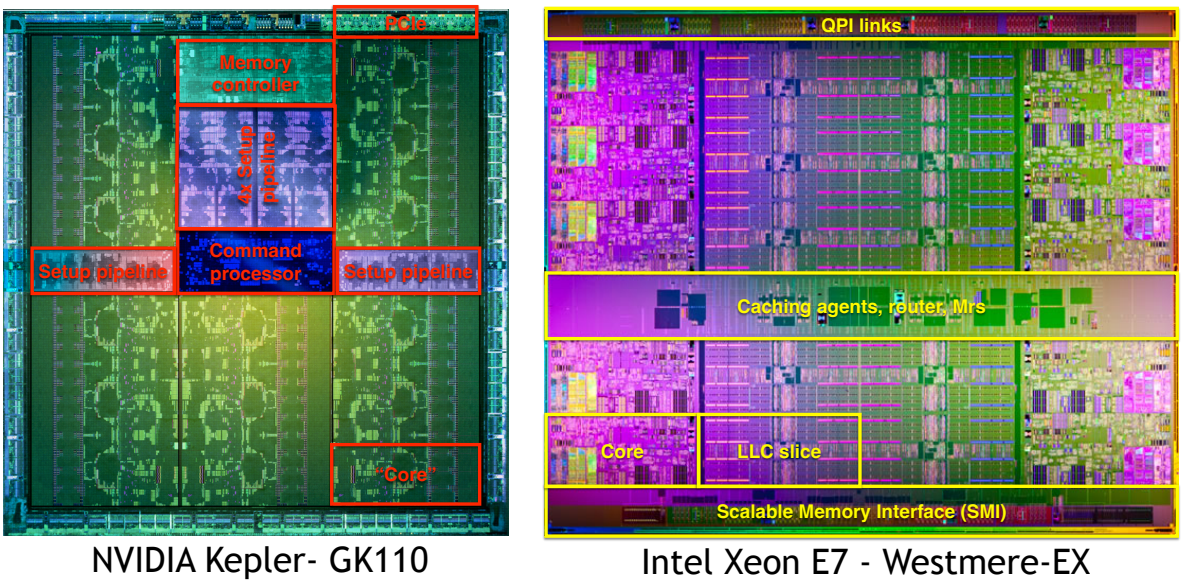


Figure 1: Die photographs: NVIDIA Kepler GK110 on the left and Intel Xeon E7 Westmere-EX on the right (slide 3). These historical examples illustrate different distributions of chip resources.

1.2 Host and device

In the model used in the lecture, the system is heterogeneous:

- the **host** is the CPU, which manages program flow;
- the **device** is the GPU, which executes highly parallel regions;

- host and device have separate memories, with explicit transfers.

Figure 2 highlights three different connections: CPU–host memory, GPU–device memory, and host–GPU. High internal GPU memory bandwidth does not make transfers from the host equally fast.

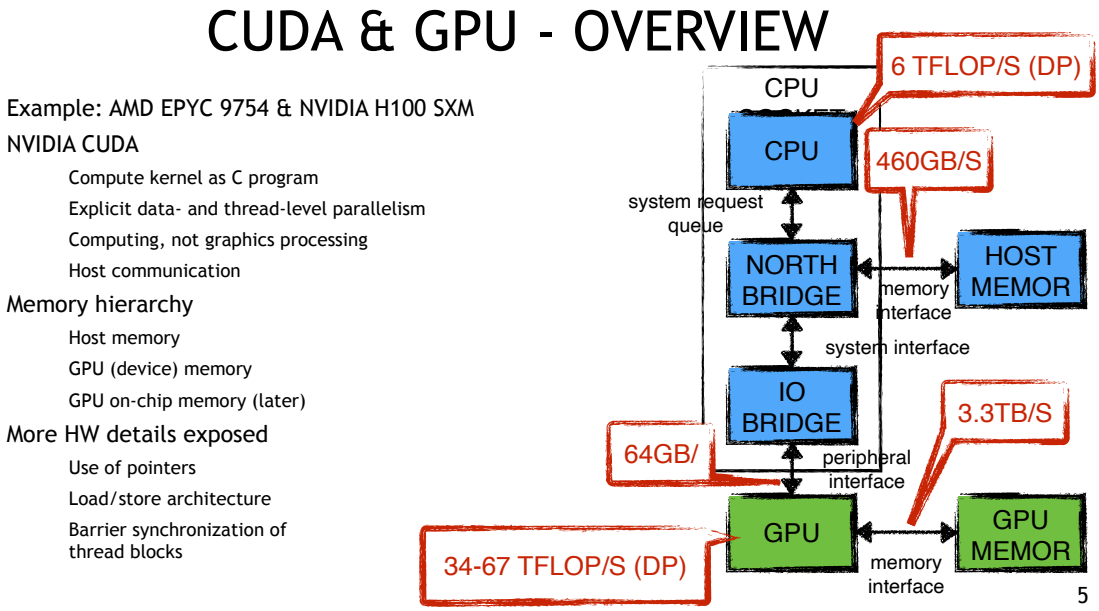


Figure 2: Host/device diagram from slide 5. The annotations provide illustrative figures for AMD EPYC 9754 and NVIDIA H100 SXM; the northbridge diagram is conceptual and does not show the detailed interconnect of those systems.

The slides quote approximately 6 TFLOP/s in double precision for the CPU, 34–67 TFLOP/s for the GPU, 460 GB/s to host memory, and 3.3 TB/s to GPU memory. These are neither SAXPY results nor guarantees for applications: performance depends on the operation type and the bottlenecks. Host–device bandwidth, annotated as 64 GB/s in the diagram, remains distinct from GPU memory bandwidth.

1.3 From the graphics pipeline to general-purpose computing

Slides 6 and 7 show the G80 architectural family from two perspectives. In graphics, work passes through stages dedicated to vertices, geometry, rasterization, and pixels. For general-purpose computing, the focus is instead on the multiprocessors that execute threads and access memory through load/store operations.

Clarification. Here, **SM** means *Streaming Multiprocessor*. The original figure also uses the abbreviation for *shared memory*; this guide always spells out the latter. An SM is a hardware unit; a CUDA block is a logical group of threads. They are not interchangeable.

2 The CUDA programming model

References: slides 9–14; handwritten notes pp. 6–7.

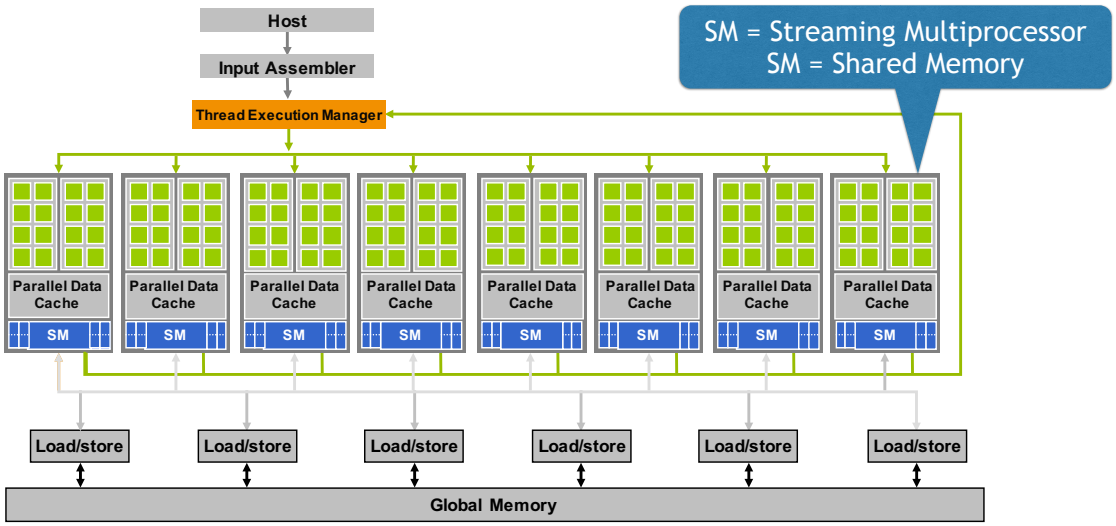


Figure 3: G80 viewed as a general-purpose computing architecture (slide 7): thread management, multiprocessors, and global memory access. This historical model helps explain CUDA abstractions.

2.1 One program, two parts

CUDA expresses GPU computation through C/C++ extensions. The CPU performs initialization, control, and work with limited parallelism; the GPU executes functions called **kernels**, applied to many threads. The lecture introduces three central abstractions:

1. a hierarchy of threads;
2. memory shared by threads within the same block;
3. barriers to synchronize those threads.

The model is **SPMD** (*Single Program, Multiple Data*): every thread executes the same kernel but uses its own index to operate on different data. Threads may follow different branches. The programmer describes scalar threads without explicitly writing vector instructions. The hardware executes threads in groups called *warps*; this connects the SPMD model to the GPU's SIMT execution.

A kernel launch is normally asynchronous with respect to the host thread: the CPU may continue before the kernel finishes. This does not mean that every copy and every kernel automatically overlap. Dependencies must be respected, with explicit synchronization where necessary.

2.2 Grids, blocks, and threads

A kernel invocation creates a **grid** consisting of blocks. Each **block** contains threads; the ordinary configuration uses blocks of the same size. Grids and blocks can have one, two, or three dimensions.

Identifier	Meaning
`threadIdx.x/y/z`	Zero-based coordinates of the thread within its block
`blockIdx.x/y/z`	Zero-based coordinates of the block within the grid
`blockDim.x/y/z`	Number of threads in the block along each axis
`gridDim.x/y/z`	Number of blocks in the grid along each axis

The `dim3` type specifies nonnegative integer dimensions; omitted components are initialized to 1. For example, `dim3 block(16,16)` describes $16 \cdot 16 \cdot 1 = 256$ threads per block. Types such as `float2`, `float4`, and `int2` group components of basic types; they do not automatically request SIMD execution.

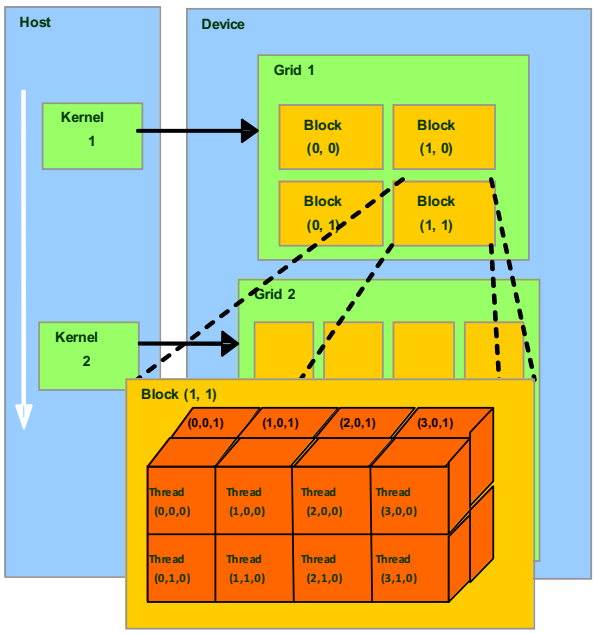


Figure 4: Hierarchy of host, kernels, grids, blocks, and threads (slide 14). Each kernel invocation creates its own grid; thread indices are local to a block.

2.3 Launch configuration and block independence

The basic syntax is:

```
kernel<<<numBlocks, threadsPerBlock>>>(args);
```

The first parameter describes the grid, the second the block. A 1D grid with G blocks and B threads per block launches GB threads. In three dimensions, the total is

$$T = (G_x G_y G_z)(B_x B_y B_z).$$

Blocks may execute in any order, concurrently or sequentially. Ordinary kernel code must not require a preceding block to have already finished. This independence allows the same grid to run on GPUs with different numbers of SMs.

Clarification. Maximum dimensions along individual axes and the maximum *total* number of threads per block are separate constraints. A tuple such as $1024 \times 1024 \times 64$ describes per-axis limits, not a valid block containing that product of threads. For the GPUs commonly considered in the lecture, the total limit is 1024; verify it on the device using `cudaGetDeviceProperties`, particularly `maxThreadsPerBlock`, `maxThreadsDim`, and `maxGridSize`. The historical limit of 65535 blocks per axis must not be treated as a universal constant either.

3 From indices to data: matrix addition

References: slides 10–13.

3.1 Why a single block is not enough

The first version in the slides maps `threadIdx.x` and `threadIdx.y` to matrix indices and uses one $N \times N$ block. This illustrates the mapping, but the block must satisfy hardware limits. Large matrices require the work to be spread across multiple blocks.

The global index in one dimension is

$$\boxed{i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}.} \tag{1}$$

With B threads per block, block b handles indices from bB to $bB + B - 1$. To cover $N > 0$ elements:

$$G = \left\lceil \frac{N}{B} \right\rceil = \frac{N + B - 1}{B} \quad \text{using integer division.} \tag{2}$$

The final block may contain extra threads: check that an index is valid before accessing the data.

3.2 A complete 2D mapping example

Here, x denotes the column and y the row. Matrices are stored in linear arrays in row-major order; element (r, c) occupies position $rN + c$. This choice keeps accesses close together for threads differing in x.

```
__global__ void matAdd(const float *A, const float *B,
                       float *C, int N)
{
    int c = blockIdx.x * blockDim.x + threadIdx.x;
    int r = blockIdx.y * blockDim.y + threadIdx.y;
    if (r < N && c < N) {
        size_t k = (size_t)r * N + c;
        C[k] = A[k] + B[k];
    }
}

// A, B, C refer to buffers already allocated on the device.
dim3 block(16, 16);
dim3 grid((N + block.x - 1) / block.x,
          (N + block.y - 1) / block.y);
matAdd<<<grid, block>>>(d_A, d_B, d_C, N);
```

For $N = 50$, each grid axis requires $\lceil 50/16 \rceil = 4$ blocks. We launch $4 \cdot 4 = 16$ blocks with 256 threads each: 4096 threads for 2500 valid elements. The remaining 1596 threads skip the addition because of the bounds check. Slide 12 shows 4.0625: this is the real-valued result of 65/16 before integer division truncates it to 4.

3.3 Choosing block size and block count

The slides suggest more blocks than SMs, approximately more than twice as many, and an indicative range of 100–1000 blocks. These are **heuristics**, not correctness requirements or a universal optimum. The aim is to provide enough ready work to tolerate latency.

A larger block contains more threads but also consumes more registers and shared memory. Heavy resource use may reduce the number of blocks that can reside simultaneously on an SM. Concurrency, resources per thread, and data volume must therefore be balanced. A practical starting point is 128 or 256 threads for a 1D kernel, followed by measurement: increasing the block size to the maximum allowed does not guarantee improvement.

4 Communication and synchronization

References: slides 14 and 18; handwritten notes p. 7.

4.1 Cooperation within a block

Threads in the same block can cooperate through shared memory, atomic operations, and barriers. The `__syncthreads()` barrier coordinates the block's threads and makes preceding shared and global memory accesses visible among them according to its guarantees. It does not synchronize the entire grid.

A typical producer–consumer sequence is:

```
__shared__ float tile[256];
int t = threadIdx.x;
tile[t] = input[t];       // each thread produces a value
__syncthreads();          // the block completes this phase
float v = tile[(t + 1) % 256]; // read another thread's value
```

This fragment assumes a 1D block with 256 threads and at least 256 valid elements in `input`. If only some threads reach the barrier, the program is incorrect: a condition surrounding the barrier must be uniform across the block. Bounds checking must not prevent some threads from participating in required barriers.

4.2 Across different blocks

Global memory is accessible to all blocks, but visibility alone does not provide synchronization. Two threads updating the same data without a correct protocol can cause a race condition. An atomic operation protects its own update; it is not a general barrier that completes every block's work.

In the lecture's basic model, the simplest way to separate dependent phases is to use two kernels in the **same stream**: the second reads the results after the first has completed. Different streams require explicit dependencies. Host synchronization, for example `cudaDeviceSynchronize()`, lets the CPU wait for preceding device work to complete.

Clarification. The statement "blocks cannot interact" in the slides should be understood as the absence of an ordinary barrier across blocks. Communication through global memory and advanced mechanisms do exist. Also, `volatile` does not replace atomics, barriers, or correct memory ordering.

5 The memory hierarchy

References: slides 15–19 and 27; handwritten notes pp. 5, 7–8.

5.1 Global memory

Global memory is device memory that all threads can read and write. It has greater capacity than on-chip memory but high latency. An explicitly allocated buffer can outlive the threads and kernels using it, until it is freed.

Fine-grained, scattered accesses can waste bandwidth. A good arrangement lets nearby threads access nearby addresses so requests can be served efficiently. This is why index mapping is also a performance decision.

The fundamental operations are:

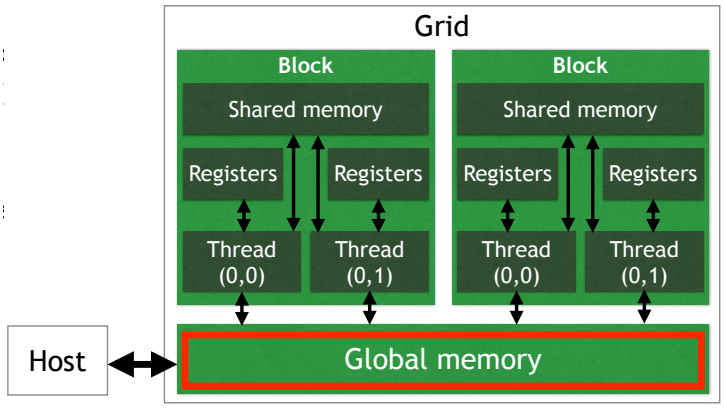


Figure 5: Visibility hierarchy (slide 15): private thread registers, shared memory per block, and global memory accessible throughout the grid. The host transfers data through the appropriate APIs.

```
float *d_x = nullptr;
size_t bytes = (size_t)N * sizeof(float);
cudaMalloc((void **)&d_x, bytes);
cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice);
// ... kernels using d_x ...
cudaMemcpy(h_x, d_x, bytes, cudaMemcpyDeviceToHost);
cudaFree(d_x);
```

`cudaMalloc` receives the address of the pointer and returns an error code; it does not directly return a buffer as `malloc` does. This corrects the pseudocode in slide 16. Pointers passed to kernels must refer to memory accessible to the device: in the explicit-memory model used here, an ordinary buffer obtained through `malloc` is not used directly as a GPU buffer.

5.2 Registers and shared memory

Registers hold private thread state. Shared memory is on-chip memory managed by the programmer and shared by threads *within the same block*. Each block has its own instance: `__shared__ float tile[256];` does not create an array shared across the entire grid.

Its lifetime is that of the **block**, not the whole program. This is more precise than the generic "thread lifetime" wording in the notes and slide 17. Shared memory supports data reuse and exchange of intermediate results, but explicit coordination is needed when a thread reads data produced by other threads.

The memory is organized into **banks**. The lecture mentions 16–32 banks with 32-bit words, depending on the architecture. In a simple model with b banks and 4-byte words:

$$\text{bank}(a) = \left\lfloor \frac{a}{4} \right\rfloor \bmod b.$$

Requests to different banks can be served in parallel. Requests to different words in the same bank may require serialization (a *bank conflict*); reads of the same word can benefit from broadcast. Access cost is not always equal to register access: conflicts and access patterns matter.

5.3 Variable qualifiers

Declaration	Location and visibility	Lifetime / use
`__device__ float v;`	Symbol in device memory, accessible to GPU code	Context lifetime; host access through symbol APIs
`__constant__ float v;`	Constant memory, read by GPU code	Context lifetime; initialized by the host through APIs
`__shared__ float v;`	Block's shared memory	Block lifetime
Kernel local variable	Private to the thread; often registers	Local execution lifetime

"Accessible from host and device" in the original table does not mean that the host can directly dereference a device symbol like a CPU variable. Appropriate operations are used, for example `cudaMemcpyToSymbol`. The texture references in slide 18 are a historical form of the texture interface; they are not needed for this guide's examples.

5.4 Pageable and pinned memory

Ordinary host memory allocated with `malloc` is *pageable*. *Pinned*, or *page-locked*, memory remains locked in physical memory and enables efficient DMA transfers. A different pair of functions allocates and frees it:

```
float *h_x = nullptr;
cudaMallocHost((void **)&h_x, bytes);
// ... use the host buffer and transfer data ...
cudaFreeHost(h_x);
```

Using `cudaMallocHost` does not move the buffer into GPU memory: it remains *host* memory. Pinned memory is a limited resource; locking too much can harm system performance. Its main benefit here concerns transfers, not kernel arithmetic.

`cudaMemcpyAsync` enqueues copies in a stream. Truly asynchronous host–device copies and useful overlap normally use pinned host memory; suitable hardware support and compatible dependencies are also needed. The word `Async` alone does not guarantee that copying and computation run concurrently. Describing `cudaMemcpy` as always "blocking" is also a simplification: behavior depends on transfer type and direction.

6 Functions and compilation

References: slides 20–22.

6.1 Function qualifiers

Qualifier	Execution	Use in the basic model
`__host__`	CPU	Function called from the host; qualifier normally implicit
`__device__`	GPU	Helper function called from device code
`__global__`	GPU	Kernel launched from the host with a grid/block configuration; returns `void`

A `__host__ __device__` function can be compiled for both environments, subject to the constraints of each.

Clarification. Absolute prohibitions in slide 20, such as "no recursion" or "no device function pointers", reflect historical limitations and should not be memorized as universal rules of current CUDA. Specific language and target restrictions still apply. For this lecture, the key distinction is where functions execute and where they are called from.

6.2 nvcc, PTX, and machine code

`nvcc` is a *compiler driver*: it coordinates the tools needed to compile the host and device portions of a CUDA program. Host code is compiled for the CPU; device code can be supplied as machine code for a target and/or as **PTX**.

PTX (*Parallel Thread Execution*) is a virtual ISA, an intermediate representation. The driver can translate it into machine code for the available GPU through *just-in-time* (JIT) compilation. This should not be confused with interpreting instructions one at a time. The CUDA runtime, `cudart`, provides the high-level interface used in the examples; the driver API is the lower-level interface.

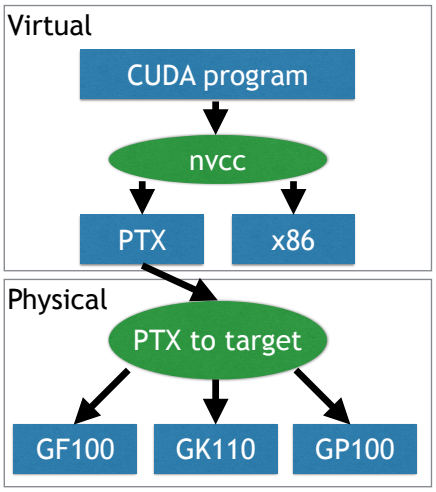


Figure 6: Conceptual compilation flow (slide 21): host and device paths, with PTX translated for a GPU target. The targets shown are historical examples.

6.3 Device properties

Slide 22 compares devices from different generations. The aim is to understand which properties to inspect, rather than treating a table as a set of constraints for all GPUs. Relevant properties include:

- compute capability, which identifies the feature family;
- number of SMs and warp size;
- available global memory;
- shared memory and registers available per block/SM;
- limits for threads, blocks, and grids;
- support for concurrent copying and execution.

`deviceQuery` is the example mentioned in the slides for inspecting these properties. The table's "CUDA cores" are not independent CPUs, and their number does not determine how many logical threads a kernel can launch.

7 SAXPY: from the algorithm to total cost

References: slides 23–27.

7.1 Operation and parallelization

SAXPY stands for *Single-precision Alpha X Plus Y*:

$$\boxed{y_i \leftarrow \alpha x_i + y_i, \qquad 0 \le i < N.} \tag{3}$$

Elements are independent provided the buffers are used without problematic overlap. The transformation is straightforward: the CPU loop becomes a set of GPU threads, each responsible for one index.

```
void saxpy_serial(int n, float alpha,
                  const float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = alpha * x[i] + y[i];
}

__global__ void saxpy(int n, float alpha,
                      const float *x, float *y)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
        y[i] = alpha * x[i] + y[i];
}
```

7.2 A minimal host sequence

The following fragment assumes $N > 0$, `h_x` and `h_y` already allocated and initialized, sizes compatible with `int` indices, and a valid device configuration. It shows the complete data lifecycle; application code must check all return values.

```
size_t bytes = (size_t)N * sizeof(float);
float *d_x = nullptr, *d_y = nullptr;
cudaMalloc((void **)&d_x, bytes);
cudaMalloc((void **)&d_y, bytes);

cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_y, h_y, bytes, cudaMemcpyHostToDevice);

int block = 256;
int grid = (N + block - 1) / block;
saxpy<<<grid, block>>>(N, alpha, d_x, d_y);
cudaError_t launchStatus = cudaGetLastError();
cudaError_t runStatus = cudaDeviceSynchronize();
// Check launchStatus and runStatus before proceeding.

cudaMemcpy(h_y, d_y, bytes, cudaMemcpyDeviceToHost);
cudaFree(d_y);
cudaFree(d_x);
```

Both x and the initial value of y are copied to the device; only the new y needs to be copied back. The scalar argument α is passed to the kernel. Explicit synchronization here makes the execution-error checkpoint clear; it is not necessary after every kernel when other operations already enforce the required dependencies.

7.3 Memory footprint and traffic

Two vectors of N four-byte `float` elements require

$$M_{\text{device}} = 2N \cdot 4 = 8N \text{ bytes.}$$

For $N = 32 \cdot 2^{20}$, this is 256 MiB on the device; the host needs the same amount if both vectors are retained there. If "32M" instead means $32 \cdot 10^6$, the result is 256 decimal MB: units should be explicit.

Each element requires two conventional floating-point operations (multiplication and addition, even when fused into an FMA), two reads, and one write. The kernel's logical traffic is therefore approximately $12N$ bytes, and its arithmetic intensity is

$$I \simeq \frac{2 \text{ FLOP}}{12 \text{ bytes}} = \frac{1}{6} \text{ FLOP/byte.}$$

This simple count, added to help interpret the example, explains why SAXPY tends to be memory-bandwidth-bound for large vectors. With effective bandwidth B_{mem}, the model's ideal limit is FLOP/s $\lesssim B_{\text{mem}}/6$. Caches, overhead, and device details can change physical traffic and observed execution time.

7.4 Kernel time and application time

If the data starts on the host, a complete measurement must include transfers:

$$T_{\text{tot}} = T_{\text{H2D}} + T_{\text{launch}} + T_{\text{kernel}} + T_{\text{D2H}},$$

plus allocations, initialization, and synchronization when these fall within the measurement scope. For one SAXPY operation, logical transfers are $8N$ bytes to the GPU and $4N$ bytes back to the host: $12N$ bytes across the host–device link.

Figure 7 shows that the kernel alone can be very fast, while including copies reduces the overall advantage. Fixed costs dominate small problems; larger problems allow the GPU to exploit parallelism. Pinned memory reduces data-movement costs but does not eliminate them.

Keeping data on the GPU across multiple kernels amortizes copying costs. If K consecutive operations reuse the buffers, one initial and one final transfer may replace transfers at every step. A meaningful measurement therefore distinguishes *kernel-only* from *end-to-end* time; the time taken for a launch call to return is not the kernel execution time.

8 Common errors and a checking method

References: slides 28–29.

`invalid configuration argument`

Check total thread count, per-axis dimensions, grid dimensions, and requested shared memory. Excessive register use or other resource requirements may also cause resource-specific launch errors; these do not all share one code.

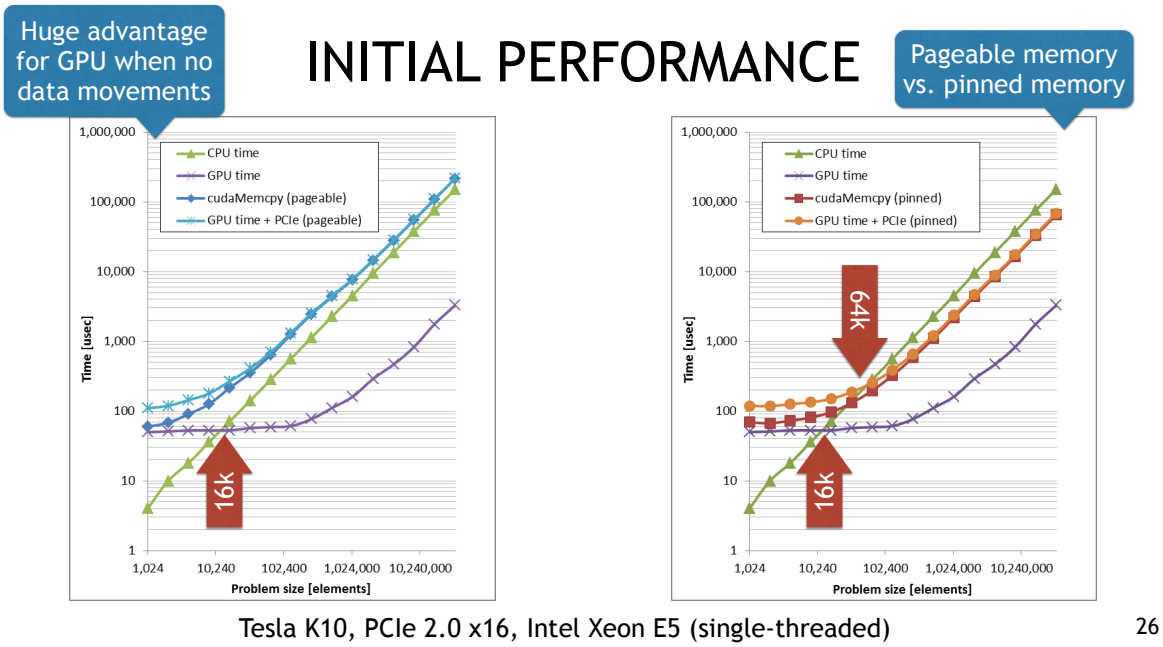


Figure 7: Measurements from slide 26: CPU time, GPU time, and transfer costs using pageable or pinned memory. Historical platform: Tesla K10, PCIe 2.0 x16, and Xeon E5 with a single-threaded CPU comparison. The highlighted thresholds belong to that experiment.

`unspecified launch failure`
: This may be associated with an illegal access or another execution error. Check indices, allocations, byte counts, and pointer validity. It does not uniquely diagnose a segmentation fault.

`the launch timed out and was terminated`
: An excessively long kernel may be interrupted by a watchdog on systems configured to use one. The slides' "Stop X11" suggestion is a historical, environment-specific workaround, not a general rule.

Shared-memory compilation errors
: Check declarations, sizes, and syntax. CUDA distinguishes static `__shared__` allocation from dynamic `extern __shared__` allocation, whose byte count is specified at launch. The `__eh_curr_region` message in the slides concerns a historical scenario; it does not mean dynamic shared memory is forbidden.

A practical method is to check every CUDA call, immediately check launch errors with `cudaGetLastError()`, and check a synchronization operation to catch asynchronous errors. Validate results against a CPU version using an appropriate numerical tolerance: floating-point arithmetic and FMA may produce small differences without an indexing error.

9 Active review

9.1 Questions with short answers

1. **Why can a GPU tolerate latency?** It can execute other ready groups of threads while some wait; enough work and resources are needed to keep those groups available.
2. **What is the difference between a block and an SM?** A block is a logical group of threads; an SM is the hardware executing assigned blocks. Multiple blocks can reside on one SM if

resources permit.

3. **With $N = 1000$ and 256 threads per block, how many blocks are needed?** Four, giving 1024 threads; 24 threads skip data accesses because of `if(i<N)`.
4. **Is a $(32, 32, 2)$ block valid with a total limit of 1024?** No: it contains 2048 threads, even if each axis satisfies its own limit.
5. **Can a block wait for block 0 to finish?** Not by assuming an ordinary scheduling order. The basic approach separates phases into kernels ordered within the same stream.
6. **Does `__syncthreads()` synchronize the host?** No: it is a barrier among a block's threads, not CPU/GPU synchronization.
7. **Does shared memory persist between kernels?** No: it belongs to a block. An explicitly allocated global buffer can persist.
8. **Is pinned memory device memory?** No: it is page-locked host memory, freed with `cudaFreeHost`.
9. **Why does SAXPY not necessarily reach peak TFLOP/s?** It performs few operations per byte transferred and is often bandwidth-bound.
10. **Why can a fast kernel still produce a slow application?** Copies, allocations, launches, and synchronization may dominate total time.

9.2 An operational sequence to remember

1. Identify independent work and the data needed on the device.
2. Allocate buffers and transfer inputs.
3. Choose block and grid dimensions valid for the device.
4. Compute global indices and check array bounds.
5. Launch the kernel and respect dependencies between phases.
6. Check errors and numerical correctness.
7. Copy results back and free each buffer with the correct API.
8. Measure total time and kernel time separately.

Sources and integration criteria

- **Course slides:** `assignment2/data/lec-02.pdf`, *GPU Computing – Architecture + Programming, Lecture 02 – CUDA Programming*, Holger Fröning and Kazem Shekofteh, Heidelberg University. Includes material from D. Kirk and W. Hwu, *Programming Massively Parallel Processors*. Figures in this guide are extracts from the cited slides.
- **Personal notes:** `assignment2/raw/GPU _261022_103341.pdf`, pp. 5–8 only (heading "Lecture 2"). Pages 1–4 cover the first lecture and have not been integrated.
- **Technical checks:** official NVIDIA documentation, *Intro to CUDA C++*, *Writing SIMT Kernels*, *C++ Language Support*, and *Runtime API: Memory Management*. Used to clarify APIs, synchronization, and historical restrictions.

This guide reorganizes and explains the materials rather than transcribing them literally. Conceptual corrections are identified in the text; thread, traffic, and arithmetic-intensity counts are educational extensions of the examples. The organizational schedule on slide 2 and divider pages are not study topics. The CUDA fragments illustrate the model; they do not constitute a standalone program compiled or benchmarked on a GPU.